Foldable & Traverse

Fold scores; traverse validated IDs

Season 6 · Functional · Intermediate

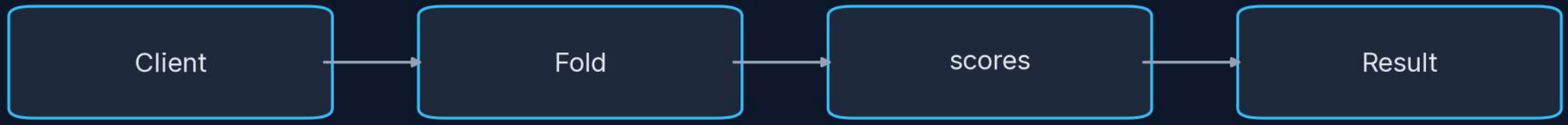
Today's Hook

- Problem: Fold scores; traverse validated IDs
- Naive pain cue: Optional.empty
- We will refactor to Foldable/Traverse with a runnable Demo.
- Failure path is locked by ProblemTest.

The Problem

- Scenario: Fold scores; traverse validated IDs
- Constraints: solve «Fold scores; traverse validated IDs» with Foldable/Traverse — keep the demo runnable and the failure path observable.
- Naive pain: Naive version fails to cleanly solve: Fold scores; traverse validated IDs
- Success looks like: 6

Structure



Foldable/Traverse: Fold scores; traverse validated IDs

Real-World Scenario

- Running example: Fold scores; traverse validated IDs
- Key types: Foldables
- Happy-path stdout cue: 6
- Failure-path stdout cue: Optional.empty

Naive Code

```java
package com.buildwithwaqar.patterns.functional.foldable.problem;

import java.util.*;

public final class NaiveFold {
  public static int sum(List<Integer> xs){ return xs.stream().mapToInt(i->i).sum(); }
}
```

Why Naive Hurts

- Naive version fails to cleanly solve: Fold scores; traverse validated IDs
- Without Foldable/Traverse, call sites absorb the wrong details.
- Failure mode to show on camera: Optional.empty
- New requirements force shotgun edits across the naive code.

Solution

```java
package com.buildwithwaqar.patterns.functional.foldable.problem;

import java.util.*;
import java.util.function.*;

final class Foldables {
  static int foldLeft(List<Integer> xs, int z, java.util.function.IntBinaryOperator op){
    int acc=z; for(int x: xs) acc=op.applyAsInt(acc,x); return acc;
  }
  static Optional<List<Integer>> traverse(List<String> ids, Function<String, Optional<Integer>> parse){
    List<Integer> out=new ArrayList<>();
    for(String id: ids){ var p=parse.apply(id); if(p.isEmpty()) return Optional.empty(); out.add(p.get()); }
    return Optional.of(out);
  }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.functional.foldable.problem;
import java.util.*; import java.util.function.*;
public final class ProblemDemo {
  public static void main(String[] args) {
    System.out.println("=== Foldable/Traverse: scores + validated IDs ===");
    System.out.println("--- naive ---"); System.out.println(NaiveFold.sum(List.of(1,2,3)));
System.out.println("--- foldable ---");
System.out.println(Foldables.foldLeft(List.of(1,2,3), 0, Integer::sum));
System.out.println(Foldables.traverse(List.of("1","2"), s -> { try{return Optional.of(Integer.parseInt(s));}catch(Exception e){return Optional.empty();} }));
System.out.println(Foldables.traverse(List.of("1","x"), s -> { try{return Optional.of(Integer.parseInt(s));}catch(Exception e){return Optional.empty();} }));
  }
}
```

Walkthrough Output

```
=== Foldable/Traverse: scores + validated IDs ===
--- naive ---
6
--- foldable ---
6
Optional[[1, 2]]
Optional.empty
```

Step-by-Step Trace

- Match stdout to stages — look for: 6
- Then the injected failure: Optional.empty
- Re-run: java -cp ... 49-foldable-traverse ProblemDemo (see episode folder).
- Pause the video and predict the next printed line.

Modern Java 25 / Zazr

```java
package com.buildwithwaqar.patterns.functional.foldable.problem;

import java.util.*;
import java.util.function.*;

final class Foldables {
  static int foldLeft(List<Integer> xs, int z, java.util.function.IntBinaryOperator op){
    int acc=z; for(int x: xs) acc=op.applyAsInt(acc,x); return acc;
  }
  static Optional<List<Integer>> traverse(List<String> ids, Function<String, Optional<Integer>> parse){
    List<Integer> out=new ArrayList<>();
    for(String id: ids){ var p=parse.apply(id); if(p.isEmpty()) return Optional.empty(); out.add(p.get()); }
    return Optional.of(out);
  }
}
```

Trade-offs

- Use Foldable/Traverse when «Fold scores; traverse validated IDs» repeats and a simpler feature does not erase it.
- Refuse extra types when a sealed switch or a function value already suffices.
- Teaching demos omit production metrics, retries, and multi-region clocks.
- Interview soundbite: state the force behind Foldable/Traverse in product language.

Related Patterns

- Name the pattern usually confused with Foldable/Traverse and separate the forces.
- Sketch the roles for «Fold scores; traverse validated IDs» from memory in sixty seconds.
- Point to the next episode in this season on the playlist map.
- Link this episode's series/ folder in the YouTube description.

Recap

- Problem: Fold scores; traverse validated IDs
- Naive pain → Foldable/Traverse boundary → Demo stdout.
- Remember the failure cue: Optional.empty
- Break a ProblemTest assertion, watch it fail, restore it.